

1 pyaudi: A truncated Taylor polynomial algebra toolbox
2 for differentiable intelligence, automatic
3 differentiation, and verified integration applications.

4 **Dario Izzo** ¹, **Francesco Biscani**², and **Sean Cowan** ¹

5 ¹ Advanced Concepts Team, European Space Research and Technology Center ² Skylon Dynamics

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#))

6 Summary

7 pyaudi is an open-source Python toolbox developed at the [European Space Agency](#) that
8 provides high-order, forward-mode automatic differentiation in a multivariate setting. The
9 toolbox is built on C++ class templates, exposed to Python via pybind11, and, at its core,
10 implements the algebra of truncated Taylor polynomials. This design allows the underlying
11 generalized dual number type to act as a seamless drop-in replacement for scalar types such
12 as floats, while operator overloading ensures that derivatives are propagated automatically.
13 The C++ code base audi can also be used directly and allows for greater flexibility in the
14 instantiation of the algebra over different fields.

15 All standard mathematical operations are supported, leveraging the nilpotency of exponentiation
16 in the algebra of truncated Taylor polynomials. In essence, within a truncated algebra $\mathcal{A}^n$,
17 any polynomial $p \in \mathcal{A}^n$ with vanishing constant term evaluates to zero when raised to a
18 power greater than the truncation order ($m > n$). This property enables efficient and exact
19 computation of derivatives to arbitrary order while maintaining the simplicity of standard
20 numerical code.

21 On top of the algebra of truncated Taylor polynomials, pyaudi also offers an implementation of
22 Taylor models ([Makino, 1998](#)), which combines truncated Taylor polynomials with an interval
23 bounding their truncation error as well as a number of miscellaneous algorithms useful for
24 applications in differential intelligence, high-order automatic differentiation, verified integration
25 and more.

26 Statement of need

27 pyaudi enables users to compute and manipulate order- n Taylor expansions of generic com-
28 putational graphs, while also providing rigorous bounds on the truncation error through their
29 associated Taylor models. These Taylor representations of program outputs can be exploited
30 in a variety of ways, including fast Monte Carlo simulations, rigorous uncertainty analysis,
31 local inversion of output–input relations, and high-order sensitivity studies. The package
32 implements the high-order automatic differentiation methodology originally developed by Berz
33 and Makino (([Berz et al., 2014](#)), ([Makino, 1998](#))), while introducing novel implementation
34 details in polynomial multiplication routines and in the bounding of Taylor models.

35 Existing libraries with similar capabilities

36 As of the time of writing, there are two main open source libraries that allow to perform similar
37 computations to those allowed by pyaudi. The first one is the C/C++ library DACE ([Massari](#)

et al., 2018) implementing the full differential algebra of truncated Taylor polynomials with float coefficients. Unlike `pyaudi`, DACE relies on a polynomial multiplication routine that makes extensive use of memory for the storage of monomial coefficients. As discussed in the comparison reported below, this approach gives DACE an advantage for single evaluations at lower orders, with the benefit diminishing as computations are performed in batches and at high orders.

A second relevant project are the Julia libraries `TaylorSeries.jl` and `TaylorModels.jl` (Benet et al., 2019) providing implementations of Taylor models to compute rigorous bounds on generic Taylor series. However, their underlying approach differs substantially from that of `pyaudi`, and preliminary comparisons presented here indicate that `pyaudi` can significantly outperform these libraries in the practical cases tested.

In the field of machine learning, in recent years a plethora of automatic differentiation toolboxes have risen tailored at the efficient execution of machine learning tasks and optimization in general. JAX (Bradbury et al., 2018), TensorFlow (Abadi et al., 2015), PyTorch (Paszke et al., 2019) (autograd) are perhaps the most widely adopted. For low order derivatives (first and second order only) these are very efficient and should be used as they implement efficiently reverse mode automatic differentiation which for these tasks is often superior. For higher order derivatives, instead, most implementation suffer greatly and, although some experimental features start to be available (see JAX's jet feature (Bettencourt et al., 2019)), they are still not mature enough.

Key aspects

The main features of `pyaudi` are:

- **Truncated polynomial algebra**, powered by `Obake`, a C++ library for symbolic manipulation of sparse multivariate polynomials, truncated power series, and Poisson series. Unlike other packages, which often suffer from severe memory bottlenecks as the polynomial order or the number of variables increases, `pyaudi` avoids large static memory allocations by adopting a sparse, dynamic approach. This remains memory-efficient at the cost of additional bookkeeping, where sparse polynomials are the area of greatest benefit. The use of templates allows to instantiate the algebra over different fields such as floats, quadruple precision floats, vectorized floats, etc..
- **Vectorized coefficients**, enabling the simultaneous evaluation of identical computational graphs at multiple expansion points. This feature makes it possible to compute high-order derivatives on multiple points, while alleviating the overhead introduced by the sparse bookkeeping of `Obake`.
- **Taylor models with Bernstein polynomial bounding**, used to enclose the range of multivariate polynomials.
- **Map inversion algorithm**, implementing the method described in (Berz et al., 2014). This feature enables the local inversion of input–output relations arising in generic computational graphs.

A performance comparison was conducted against DACE for `gduals`, and against `TaylorModels.jl` for Taylor models, of which the results can be found in `paper_results.md` on the paper branch.

Ongoing research

(Acciarini et al., 2024) extended the principle of numerical continuation to find solutions in non-linear dynamical systems in the space domain to the moments of a probability density function, using `pyaudi` to calculate the high-order derivatives in the Circular Restricted Three-Body Problem (CR3BP). Similarly, `pyaudi` was used in (Acciarini et al., 2025) for various test cases involving non-Gaussian uncertainty distributions to calculate the high-order derivatives. (Caleb

85 & Lizy-Destrez, 2020) did work on exploring Differential Algebra (DA) for accelerating Monte
 86 Carlo simulations, which is enabled by libraries such as pyaudi. For on-board applications,
 87 pyaudi was used in (Burnett & Topputo, 2025) to implement convex guidance algorithms by
 88 describing the solutions using DA. (Izzo et al., 2017) explored Cartesian Genetic Programming
 89 and made it differentiable with high-order automatic differentiation, enabled by pyaudi.

90 Acknowledgement of financial support

91 No financial support was provided for the development of this software.

92 Bibliography

- 93 Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis,
 94 A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia,
 95 Y., Jozefowicz, R., Kaiser, L., Kudlur, M., ... Zheng, X. (2015). *TensorFlow: Large-scale*
 96 *machine learning on heterogeneous systems*. <https://www.tensorflow.org/>
- 97 Acciarini, G., Baresi, N., Lloyd, D. J., & Izzo, D. (2024). Stochastic continuation of trajectories
 98 in the circular restricted three-body problem via differential algebra. *arXiv Preprint*
 99 *arXiv:2405.18909*.
- 100 Acciarini, G., Baresi, N., Lloyd, D. J., & Izzo, D. (2025). Nonlinear propagation of non-gaussian
 101 uncertainties. *Journal of Guidance, Control, and Dynamics*, 48(4), 903–913.
- 102 Benet, L., Forets, M., Sanders, D., & Schilling, C. (2019). TaylorModels. JI: Taylor models in
 103 julia and their application to validated solutions of ODEs. In *SWIM* (pp. 15–16).
- 104 Berz, M., Makino, K., & Wan, W. (2014). *An introduction to beam physics*. Taylor & Francis.
- 105 Bettencourt, J., Johnson, M. J., & Duvenaud, D. (2019). Taylor-mode automatic differentiation
 106 for higher-order derivatives in JAX. *Program Transformations for ML Workshop at NeurIPS*
 107 *2019*.
- 108 Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G.,
 109 Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable*
 110 *transformations of Python+NumPy programs* (Version 0.3.13). [http://github.com/jax-ml/](http://github.com/jax-ml/jax)
 111 [jax](http://github.com/jax-ml/jax)
- 112 Burnett, E. R., & Topputo, F. (2025). Rapid nonlinear convex guidance using a monomial
 113 method. *Journal of Guidance, Control, and Dynamics*, 48(4), 736–756. [https://doi.org/](https://doi.org/10.2514/1.G008512)
 114 [10.2514/1.G008512](https://doi.org/10.2514/1.G008512)
- 115 Caleb, T., & Lizy-Destrez, S. (2020). Can uncertainty propagation solve the mysterious case of
 116 snoopy? *International Conference on Uncertainty Quantification & Optimisation*, 109–128.
 117 https://doi.org/10.1007/978-3-030-80542-5_8
- 118 Izzo, D., Biscani, F., & Mereta, A. (2017). Differentiable genetic programming. *European*
 119 *Conference on Genetic Programming*, 35–51. <https://doi.org/10.1007/978-3-319-55696-3>
- 120 Makino, K. (1998). *Rigorous analysis of nonlinear motion in particle accelerators* [PhD thesis].
 121 Michigan State University.
- 122 Massari, M., Di Lizia, P., Cavenago, F., & Wittig, A. (2018). Differential algebra software library
 123 with automatic code generation for space embedded applications. In *2018 AIAA information*
 124 *systems-AIAA infotech@ aerospace* (p. 0398). <https://doi.org/10.2514/6.2018-0398>
- 125 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin,
 126 Z., Gimelshein, N., Antiga, L., & others. (2019). Pytorch: An imperative style, high-
 127 performance deep learning library. *Advances in Neural Information Processing Systems*,

DRAFT